

What If Your AI Never Had to Guess the Format?

Jev is a new kind of AI model — built to return typed decisions, not text. Could it change how you design software that thinks?





Who Built Jev — and Why

TypeSafe AI launched September 15, 2026, with \$40M seed funding led by DCVC. Founder Diogo Almeida co-invented RLHF and InstructGPT at OpenAI. One mission: build AI designed for machines, not conversations.

A photograph of a server rack filled with various network and computing hardware. Numerous blue and yellow Ethernet cables are plugged into the front panels of the units, creating a complex web of connections. The rack is black and the units are also black, with some green status lights visible.

The Problem With Using LLMs for Decisions

LLMs produce text for humans to read. When your code needs a decision, you coerce the LLM into returning JSON — then parse it back out. That mismatch creates fragility, latency, and cost. Jev eliminates it: structured input in, typed decision out, usable directly by code.

System 1 vs. System 2 — The Core Idea

System 1

Daniel Kahneman: fast, automatic thinking is System 1. Most software decisions are System 1 — bounded, repeated, fast. Jev is designed specifically for the System 1 layer.

System 2

Slow, deliberate reasoning is System 2. LLMs are System 2. Overbuilt and expensive for that job.

Jev's Position — Speed and Cost vs. LLMs

TypeSafe reports up to 193.6x faster and 444.6x cheaper than comparable LLMs on classification tasks. Input priced at \$0.042 per million tokens. Output tokens are free. These are TypeSafe's own vendor-reported benchmark figures — treat them as directional.

Read the launch post: <https://typesafe.ai/blog/introducing-system-one-models-and-jev>



A vertical wooden door with a large glass panel. The glass is slightly out of focus, showing a glimpse of another room with a window and a wooden floor. The door has a silver handle and hinges.

What Jev Cannot Do

Jev cannot generate text, write replies, or hold a conversation. It cannot explain its reasoning or perform arithmetic. Its limits are structural — not a gap to be patched in a future version. If the output is something a human reads directly, that job belongs to an LLM.

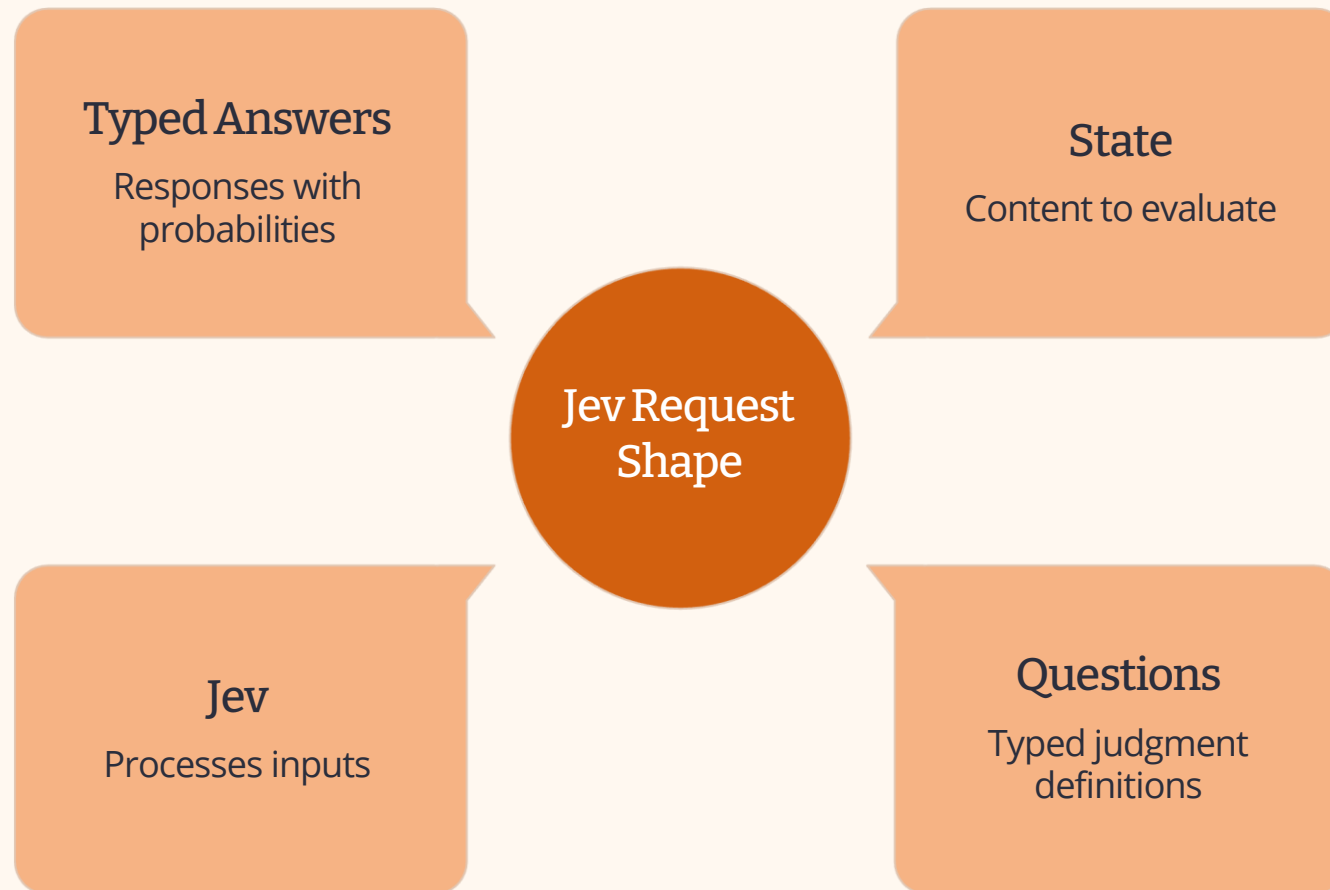
Check In With Your Instructor

Think about what you just learned about System 1 vs. System 2. Tell your instructor in your own words: what kind of software decision is Jev built for — and what kind is it not built for? Your instructor will confirm or gently correct your answer before we move on.



The Shape of Every Jev Request

Every Jev call has two inputs: state and questions. State is the content to evaluate — a string, a JSON object, or an array of text. Questions are typed definitions of the judgments you want made. Jev returns one typed answer per question, with probabilities. No text parsing required.



The Three Primitives

Choice

Pick one option from a list you define.
Returns the winner, all probabilities, and confidence.

Score

Place state on a 2–10 level rubric you define in words. Returns a fractional score, per-level probabilities, and confidence.

Noul

Yes or no. Returns a single probability from 0 to 1 that the answer is yes. No separate confidence field needed.

Confidence — What It Means and How to Use It

Confidence reflects how concentrated the probability distribution is — not whether the answer is factually correct.

Above ~0.85

Act automatically.

0.5 to 0.85

Flag for human review or gather more data.

Below 0.5

Escalate to a human or a larger model.



One Jev Call — A Support Ticket in Action

A customer writes: "I've been trying to connect my Stripe account for three days. I'm losing sales. Please help ASAP."

One Jev call returns three answers simultaneously:

Department (Choice)

= technical, 85% probability, confidence
0.78

Frustration (Score)

= 1.0 on a three-level rubric, confidence
1.0

Is_urgent (Noul)

= 0.91

Code branches directly on these typed values — no parsing required. <https://docs.typesafe.ai/introduction/quickstart>



Primitive Identification Drill

Before we move on, let's make sure the three primitives are solid. Ask your instructor to run the primitive drill with you. Your instructor will describe three situations one at a time — you name which primitive fits and explain why.

What Type Safety Guarantees

Type safety means Jev always returns a value that matches the schema you defined.

Choice

A Choice answer will always be one of your listed options.

Score

A Score will always be within your defined range.

Noul

A Noul will always be a probability from 0 to 1.

TypeSafe claims a 0% structured-output type error rate — a structural guarantee from schema design, not an empirical measurement against a test set.

What Type Safety Does Not Guarantee

Jev can still select the wrong valid option with high confidence. That is misclassification — not hallucination. The answer will be well-formed. It may still be wrong. Confidence tells you how to route a decision, not whether the decision is factually correct.



When Jev Fits — The General Pattern

Jev fits when the possible outcomes are known in advance and can be listed. The decision is made many times, at volume, and the result feeds directly into code. Speed and cost matter.

 The plain test: if a step's output would be parsed from LLM text into a category, score, or yes/no anyway — it belongs in Jev.

Jev and LLMs Working Together

Jev

Jev is not a replacement for an LLM. It is a fast control plane around one. Jev handles routing, gating, and scoring.

LLM

The LLM handles the open-ended generation and reasoning that follows. If the output is something a human reads directly — a reply, an explanation, a generated artifact — that step belongs in the LLM.



Fit-or-Not-Fit Exercise

Now let's test your judgment on real situations. Ask your instructor to run the fit-or-not-fit exercise with you. For each scenario your instructor describes, say aloud whether it is a Jev fit, an LLM fit, or a hybrid — and give one reason why.

Three Ways to Access Jev

1 TypeSafe's own API

Sign up at typesafe.ai and join the early-access waitlist.

2 Third-party gateways with no TypeSafe waitlist required

Cloudflare Workers AI (model ID: `typesafe/jev`), Vercel AI Gateway, and OpenRouter.

3 Two official SDKs

Python at github.com/typesafe-ai/typesafe-sdk-python and JavaScript/TypeScript at github.com/typesafe-ai/typesafe-sdk-js. Set your API key as the environment variable `TYPESAFE_API_KEY` — the SDK reads it automatically.

Two Production Habits That Matter Most

For development, use `jev-latest` for convenience. For production, pin to a specific versioned model ID such as `jev-1.13.0`. The `jev-latest` alias moves when TypeSafe ships a new version — which can silently change model behavior. The single most important habit: log the model field from every response so you always know which version answered.



Other Best Practices Worth Knowing

- Always include an "other" option in Choice questions so Jev can signal when nothing fits.
- Send all the questions you need in one request — they run in parallel, so adding questions barely changes latency or cost.
- Send only the state the questions need — extra context lowers accuracy.

Ask your instructor to clarify any access path, SDK, or best practice before we close.



Optional — Test Your Own Understanding

A private knowledge check is available if you want to confirm you have the core ideas straight. It covers the primitives, confidence zones, type safety limits, and the hybrid architecture pattern. Nothing is recorded. It is just for you. Ask your instructor to begin the knowledge check whenever you are ready.



You Are Ready to Work With Jev

You can now explain what Jev is, read its three primitives and a full response, and judge whether a use case fits Jev or needs an LLM instead.

Your next step: visit [typesafe.ai](https://docs.typesafe.ai/introduction/quickstart), join the waitlist, and try the official quickstart at docs.typesafe.ai/introduction/quickstart.

Thank you for taking this course.